

ASSIGNMENT 1

Ques 1. What is Full Stack Development? Explain the term Full Stack Development and the role of a Full Stack Developer.

Solution: Full Stack Development is the process of developing both the **front-end (client side)** and **back-end (server side)** parts of a web application. A full-stack application includes everything from the user interface that users interact with to the databases and servers that store and process data.

⇒ Main Components of Full Stack Development

1. Front-End Development

- The visual part of a website or application.
- Responsible for user interaction and design.
- Common technologies:
 - HTML
 - CSS
 - JavaScript
 - Frameworks/Libraries such as React, Angular, or Vue.js

2. Back-End Development

- Handles business logic, authentication, and data processing.
- Communicates with databases and serves data to the front end.
- Common technologies:
 - Node.js
 - Python
 - Java
 - PHP
 - .NET

3. Database Management

- Stores and manages application data.
- Common databases:
 - MySQL

- PostgreSQL
- MongoDB
- SQLite

4. Version Control and Deployment

- Managing source code and deploying applications.
- Common tools:
 - Git
 - GitHub
 - Docker
 - Cloud platforms such as AWS, Microsoft Azure, and Google Cloud

⇒ Role of a Full Stack Developer

A **Full Stack Developer** is a software professional who can work on both the front-end and back-end of an application. They understand the complete development process and can build an entire web application from start to finish.

Responsibilities of a Full Stack Developer

- Designing and developing user interfaces.
- Creating server-side logic and APIs.
- Managing databases and data storage.
- Integrating front-end and back-end components.
- Testing and debugging applications.
- Ensuring security and performance.
- Deploying and maintaining applications.
- Collaborating with designers, developers, and stakeholders.

Ques 2. What is Frontend? Explain Frontend and list some technologies used to build it.

Solution: Frontend (or Front-end development) refers to the **client-side part of a website or web application**—everything that users directly see and interact with in their browser.

It includes the layout, design, buttons, menus, images, animations, and overall user experience (UI/UX). The frontend is responsible for making a website **visually appealing, responsive, and interactive**.

⇒ **What Does a Frontend Developer Do?**

A frontend developer:

- Builds user interfaces (UI)
- Converts designs into working web pages
- Ensures websites work on different devices (mobile, tablet, desktop)
- Improves performance and user experience
- Handles user interactions (clicks, forms, animations, etc.)

⇒ **Technologies Used in Frontend Development**

Frontend development mainly relies on these technologies:

1. Core Technologies

- **HTML (HyperText Markup Language)** – Structure of web pages
- **CSS (Cascading Style Sheets)** – Styling and layout
- **JavaScript** – Adds interactivity and dynamic behavior

2. Frontend Frameworks & Libraries

These make development faster and more organized:

- **React.js** – Popular library for building user interfaces
- **Angular** – Full-featured frontend framework
- **Vue.js** – Lightweight and flexible framework
- **Svelte** – Modern framework focused on performance

3. Styling Tools

- **Bootstrap** – Pre-built responsive UI components
- **Tailwind CSS** – Utility-first CSS framework
- **Sass/SCSS** – CSS preprocessor for advanced styling

4. Tools & Supporting Technologies

- **TypeScript** – Superset of JavaScript with type safety
- **Webpack / Vite** – Module bundlers and build tools

- **Figma / Adobe XD** – UI design tools used before coding

Ques3. What is Backend? Explain Backend and mention its main responsibilities.

Solution: Backend refers to the *server-side* part of a software application that users don't directly see or interact with. It is responsible for managing how the application works behind the scenes—handling data, logic, and communication between systems.

Think of a simple example: when you log into an app, the backend checks your username and password, retrieves your data, and sends the correct response to your screen.

=> **Main Responsibilities of Backend**

1. Server-side logic

The backend contains the core rules and logic of an application.

- Processes user requests
- Applies business rules (e.g., calculations, validations)
- Decides what data to send back

2. Database management

It handles storing, retrieving, updating, and deleting data.

- Works with databases like MySQL, PostgreSQL, MongoDB, etc.
- Ensures data is stored securely and efficiently

3. API handling (communication layer)

Backend exposes APIs (Application Programming Interfaces) so the frontend can talk to it.

- Receives requests from frontend (e.g., "get user profile")
- Sends back responses in formats like JSON

4. Authentication & Authorization

Ensures security by controlling who can access what.

- Login/logout systems
- Password verification
- Role-based access (admin, user, etc.)

5. Performance & scalability

Ensures the system runs smoothly even with many users.

- Load balancing
- Caching frequently used data
- Optimizing queries and server performance

6. Security

Protects the application and user data.

- Encrypts sensitive data
- Prevents attacks like SQL injection or unauthorized access
- Validates inputs

Ques4. What is a Database? Why do websites need databases? Give examples of data stored in a database.

Solution:

A **database** is an organized system used to **store, manage, and retrieve data efficiently**. It acts like a digital storage room where information is kept in a structured way so that applications can easily access and update it.

Databases are usually managed by software called a **Database Management System (DBMS)**, such as MySQL, PostgreSQL, or MongoDB.

=> Why do websites need databases?

Websites need databases because most modern websites are **dynamic**, meaning they handle constantly changing user data.

Here's why databases are essential:

1. Store user data

Websites need to remember users and their information.

- Accounts, passwords, profiles, preferences

2. Save and retrieve information

Websites must fetch data when needed.

- Showing your posts on social media
- Loading your shopping cart items

3. Support interactions

Websites are interactive and need to track actions.

- Likes, comments, messages
- Form submissions

4. Ensure data consistency

Databases keep data accurate and up-to-date.

- If you update your profile picture, it changes everywhere instantly

5. Handle large-scale data

Websites like YouTube or Amazon manage millions of users and huge amounts of data efficiently.

⇒ **Examples of data stored in a database**

Different websites store different types of data. Here are common examples:

User data

- Name
- Email address
- Password (encrypted)
- Phone number

E-commerce data

- Product details (price, description, images)
- Orders and payment history
- Shopping cart items

Social media data

- Posts and captions
- Comments and likes
- Friend/follow lists

Streaming platforms

- Watch history
- Video metadata (title, duration, views)
- User subscriptions

Banking apps

- Account balance
- Transaction history
- Beneficiary details

Ques 5. What is DNS? Explain how DNS helps users access websites.

Solution: DNS (Domain Name System) is like the **phonebook of the internet**. It translates human-friendly website names (like google.com) into machine-friendly IP addresses (like 142.250.190.14) that computers use to find each other.

⇒ **Why DNS is needed**

Computers communicate using **IP addresses**, but humans find numbers hard to remember. DNS solves this problem by letting us use easy names instead.

For example:

- You type: youtube.com
- DNS finds: 142.250.xxx.xxx
- Your browser connects to that IP address

=> **How DNS helps users access websites (step-by-step)**

1. You enter a website name

You type a URL like:

www.wikipedia.org

2. Browser asks DNS

Your browser asks a DNS server:

“What is the IP address of this website?”

3. DNS lookup happens

The DNS system checks different levels:

- Cache (your device or browser memory)
- Local DNS server (ISP)
- Root DNS servers
- Authoritative DNS server

It eventually finds the correct IP address.

4. DNS returns the IP address

It sends back something like:

208.80.154.224

5. Browser connects to the website

Now your browser uses that IP address to:

- Connect to the web server
- Request website data
- Load the page on your screen

Ques6. What is an IP Address? Why are IP addresses important in networking?

Solution: An **IP address (Internet Protocol address)** is a unique number assigned to every device connected to a network (like the internet). It acts as an **identifier and location address** for devices so they can communicate with each other.

Example of an IP address:

- IPv4: 192.168.1.1
- IPv6: 2001:0db8:85a3::8a2e:0370:7334

⇒ **Why are IP addresses important in networking?**

IP addresses are essential because they make communication over networks possible.

1. Device identification

Every device needs a unique address to be recognized on a network.

- Like a house number for your device

2. Data routing (sending information)

IP addresses help direct data to the correct destination.

- When you open a website, data is sent to *your* IP address, not someone else's

3. Communication between devices

Devices use IP addresses to exchange information.

- Phones, laptops, servers all communicate using IPs

4. Accessing websites and services

Websites are hosted on servers with IP addresses.

- Your browser uses the IP address to load websites

5. Network management and security

Networks use IPs to:

- Monitor traffic
- Block suspicious devices
- Control access (firewalls, filters)

Ques7. What is Client-Server Architecture? Explain how a client and server communicate with each other.

Solution: Client–Server Architecture is a computing model where tasks are divided between two types of systems:

- **Client:** The device or application that *requests services or data*
- **Server:** The system that *provides services or data*

This model is the foundation of most modern applications like websites, mobile apps, and online services.

Examples:

- Your browser (Chrome) = client
- Google’s computer systems = server

⇒ **How Client and Server Communicate**

Communication happens in a structured request–response cycle:

1. Client sends a request

The client asks for something.

- Example: You type `www.youtube.com`
- Your browser (client) sends a request to the server

2. Request travels over the network

The request is sent using the internet, often through protocols like:

- HTTP / HTTPS

3. Server processes the request

The server:

- Receives the request
- Processes it (checks database, runs logic, etc.)
- Prepares a response

4. Server sends a response

The server sends back data such as:

- Web page content
- Images, videos, or JSON data

5. Client displays the result

The client (browser/app):

- Interprets the response
- Displays the website or data to the user

⇒ **Simple Example**

When you open Instagram:

1. Client → “Show my feed”
2. Server → Fetches posts from database
3. Server → Sends posts back
4. Client → Displays feed on your screen

Ques8. What is an API? Explain the role of APIs in web applications.

Solution: An **API (Application Programming Interface)** is a set of rules that allows different software applications to **communicate with each other**.

In simple terms, an API is like a **messenger** that takes a request from one program, sends it to another, and returns the response.

Example:

- Your app → asks for weather data
- API → gets data from weather server
- API → sends data back to your app

⇒ **Role of APIs in Web Applications**

APIs are the **bridge between frontend (client) and backend (server)** in most web applications.

1. Communication between client and server

APIs allow the frontend to talk to the backend.

- Frontend sends request (e.g., “get user profile”)
- Backend responds using API

2. Data exchange

APIs send and receive data, usually in formats like **JSON**.

- Example: user details, posts, products

3. Access to services and features

Web apps use APIs to access external services:

- Payment gateways (Razorpay, Stripe)
- Maps (Google Maps API)
- Social login (Google/Facebook login)

4. Modular development

APIs allow systems to be built in parts.

- Frontend and backend can be developed separately
- Easy to update or scale components

5. Integration of third-party tools

APIs help websites use external features without building them from scratch.

- Weather apps use weather APIs
- E-commerce sites use shipping APIs

Ques9. What are HTTP Methods? Explain the purpose of:

- GET
- POST
- PUT
- DELETE

Solution: HTTP Methods are actions used in web communication to tell a server what operation to perform on data. They are commonly used in APIs and web applications.

1. GET

Used to **retrieve data** from the server.

- It does not change anything on the server
- Only reads data
- Most common method

Example:

- Loading a webpage
- Getting user details or product list

2. POST

Used to **send data to the server to create new data**.

- Adds new information to the database
- Commonly used in forms
- Can create duplicates if repeated

Example:

- User signup
- Submitting a form
- Placing an order

3. PUT

Used to **update or replace existing data**.

- Updates an existing resource completely
- If the same request is repeated, result stays the same (idempotent)

Example:

- Updating user profile (name, email, etc.)
- Replacing product details

4.DELETE

Used to **remove data from the server**.

- Deletes a specific resource
- Once deleted, it is removed from database

Example:

- Deleting a user account
- Removing a post or product

Ques10. What are Developer Tools? Name any three tabs available in Developer Tools and explain their purpose.

Solution: Developer Tools (DevTools) are built-in tools in web browsers like Chrome, Firefox, and Edge that help developers **inspect, debug, and test web pages**.

They allow you to see how a website works behind the scenes, fix errors, and analyze performance.

You can open them by pressing:

- **F12** or **Ctrl + Shift + I**

=> **Three important tabs in Developer Tools**

1. Elements Tab

This tab shows the **HTML structure and CSS styles** of a webpage.

- You can inspect and edit page layout in real time
- Helps understand how the UI is built
- Useful for fixing design issues

Example: Changing button color or text temporarily

2. Console Tab

This tab is used to **view errors, logs, and run JavaScript code**.

- Shows website errors and warnings
- Helps in debugging code
- You can test JavaScript commands directly

Example: Finding “undefined” errors or testing scripts

3. Network Tab

This tab shows all **network requests made by the website**.

- Displays API calls, images, and file loading
- Helps check if data is loading correctly
- Useful for debugging backend/API issues

Example: Checking if a login API request is working

Ques11. Draw the Full Stack Architecture Diagram. User ↓ Frontend ↓ Backend ↓ Database
Write a short description of each layer.

Solution: Full Stack Architecture Diagram;

User



Frontend



Backend



Database

1. User

The **user** is the person who interacts with the application using a browser or mobile app.

- Clicks buttons, fills forms, views pages
- Sends requests like login, search, or purchase

2. Frontend

The **frontend** is the visible part of the application (client-side).

- Built using HTML, CSS, JavaScript, or frameworks like React
- Displays UI (buttons, pages, forms)
- Sends user requests to the backend
- Shows responses received from backend

Example: Login page, homepage, dashboard

3. Backend

The **backend** is the server-side logic that processes requests.

- Handles business logic
- Processes user requests from frontend
- Communicates with database
- Sends responses back to frontend

Example: Login verification, data processing, API handling

4. Database

The **database** stores and manages all application data.

- Stores user info, posts, products, etc.
- Allows create, read, update, delete (CRUD) operations
- Ensures data is safe and organized

Example: User accounts, order history, posts

Ques12. Draw the Complete Website Working Flow.

User



Browser



DNS



Server



Database



Server



Browser



Website Displayed

Explain each step in one sentence.

Solution: Complete Website Working Flow;

User



Browser



DNS



Server



Database



Server



Browser



Website Displayed

- **User** → The user enters a website URL or clicks a link to access a website.
- **Browser** → The browser processes the request and prepares to find the website's server.
- **DNS** → DNS converts the domain name into an IP address of the server.
- **Server** → The server receives the request and processes it using backend logic.
- **Database** → The database provides the required stored data to the server.
- **Server** → The server sends the processed data and response back to the browser.
- **Browser** → The browser receives the response and converts it into a visual webpage.
- **Website Displayed** → The final website is shown to the user on the screen.

Ques13. Student Registration System A student fills a registration form on a website. Explain:

- What happens in the Frontend?
- What happens in the Backend?

- What happens in the Database?

Solution: When a student fills a registration form on a website, the system works across **Frontend, Backend, and Database.**

⇒ **Frontend**

- The student fills in details like name, email, password, and course in a form.
- The frontend checks basic validation (like required fields or email format).
- When the student clicks “Submit,” the frontend sends the data to the backend using an API request.
- It shows messages like “Submitting...” or “Registration successful.”

⇒ **Backend**

- The backend receives the form data from the frontend.
- It checks if the data is valid and secure (e.g., password strength, duplicate email).
- It processes the request and applies business rules.
- Then it prepares the data to be stored in the database.
- Finally, it sends a response back to the frontend (success or error message).

⇒ **Database**

- The database stores the student’s information permanently.
- It saves data like name, email, password (encrypted), and course details.
- It ensures data is organized and can be retrieved later.
- If needed, it can also update or prevent duplicate entries.

Ques14. Amazon Search Scenario A user searches for "Laptop" on Amazon. Explain the complete flow from entering the search term to displaying the results. Use the following terms in your answer: • Browser • Server • Database • Response

Solution: When a user searches for “Laptop” on Amazon, the system follows a complete flow:

⇒ **Complete Flow Explanation**

1. Browser

The user types “**Laptop**” in the search bar and presses enter.

The **browser** sends this search request to Amazon’s server using the internet.

2. Server

The **server** receives the request and processes it.

It understands that the user is searching for products related to “Laptop” and prepares a query to fetch matching items.

3. Database

The server sends a request to the **database**, which stores all product information.

The database searches for laptops based on name, category, price, and other details, and finds matching results.

4. Response

The database sends the matching data back to the **server**, and the server formats it into a proper response (usually JSON).

Then the server sends this **response** back to the browser.

5. Final Display

The browser receives the response and displays the list of laptops with images, prices, ratings, and details to the user.

Ques15. Website Observation Activity Open any website and inspect it using Developer Tools (F12). Write:

- Website Name
- One observation from the Elements Tab
- One observation from the Network Tab
- One observation from the Console Tab

Solution: Website Name: Amazon (www.amazon.com)

⇒ **Elements Tab Observation:**

The Elements tab shows the **HTML structure of the page**, where you can see product sections, headings, images, and buttons arranged in a structured layout.

⇒ **Network Tab Observation:**

The Network tab shows that when the page loads or when you search something, the browser makes **multiple API requests to load products, images, and recommendations**.

⇒ **Console Tab Observation:**

The Console tab may show warnings or errors, such as failed scripts, blocked resources, or performance-related messages from the website.

Ques16. In your own words, explain how a website works from the moment a user enters a URL until the webpage appears on the screen.

Solution: When a user enters a URL in the browser, the process of loading a website starts in a series of steps behind the scenes.

First, the **user types a URL** (like `www.example.com`) in the browser. The **browser** checks whether it already knows the website's IP address from its cache. If not, it sends a request to the **DNS (Domain Name System)**, which converts the domain name into the correct **IP address** of the server.

After getting the IP address, the browser connects to the **server** where the website is hosted and sends an HTTP/HTTPS request asking for the webpage.

The **server** receives this request and processes it. If the website needs data such as user details, posts, or products, it communicates with the **database** to fetch the required information.

The **database** retrieves and sends the requested data back to the server. The server then combines this data with website files (HTML, CSS, JavaScript) and creates a response.

Finally, the server sends this **response** back to the **browser**, which reads the code and renders it into a visual webpage. The user then sees the fully loaded website on the screen.

Ques17. Explain the difference between Frontend, Backend, and Database using a real-world example of your choice (Amazon, Instagram, Zomato, Netflix, etc.).

Solution: Let's take **Amazon** as a real-world example to understand the difference between **Frontend, Backend, and Database**.

⇒ **Frontend (What the user sees)**

The **frontend** is the visible part of Amazon that users interact with.

On Amazon, the frontend includes:

- Home page with product listings
- Search bar
- Product images, prices, and ratings

- “Add to Cart” and “Buy Now” buttons

It is built using HTML, CSS, JavaScript, and frameworks like React.

In short: Frontend = everything the user sees and clicks.

⇒ **Backend (How it works behind the scenes)**

The **backend** is the brain of Amazon that handles all logic and processing.

On Amazon, the backend:

- Processes search requests like “laptop”
- Checks user login and authentication
- Calculates prices, discounts, and delivery options
- Handles orders and payments

It works on servers using languages like Java, Python, Node.js, etc.

In short: Backend = processes requests and runs the system logic.

⇒ **Database (Where data is stored)**

The **database** stores all Amazon data in an organized way.

On Amazon, the database stores:

- User accounts and passwords (encrypted)
- Product details (name, price, stock, images)
- Orders and payment history
- Reviews and ratings
- It ensures data is stored safely and can be retrieved quickly.

In short: Database = stores all information permanently.

⇒ **Real-life flow example (Amazon search “Laptop”)**

1. **Frontend** → User types “Laptop” and clicks search
2. **Backend** → Receives request and processes it
3. **Database** → Finds matching laptop products
4. **Backend** → Sends results back
5. **Frontend** → Displays products on screen

